

Built-in Packages and Imports

An import lets a source file use a simple type name; package declarations supply identity, and access modifiers supply visibility.

This complete offline chapter connects ideas, diagrams, exact traces, commented Java and exam practice. Switch themes for reading; printing always uses the light presentation and includes every required trace and answer.

GUIDE 2026-10-07.1 · STANDALONE HTML AND PDF SHARE THE SAME CONTENT

1. The problem and the plain-language idea

An import lets a source file use a simple type name; package declarations supply identity, and access modifiers supply visibility.



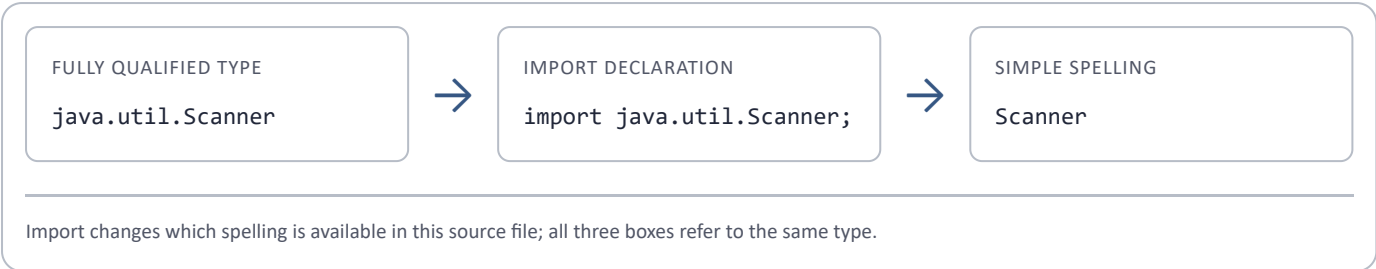
Think of a package as a class address. Two classes may share a simple name when their package addresses differ. Source paths and compiled paths are measured from different roots.

Source root: src/	Output / classpath root: classes/
src/	classes/
pkg/	pkg/
A.java	A.class
B.java	B.class
Tester.java	Tester.class
app/	app/
Main.java	Main.class

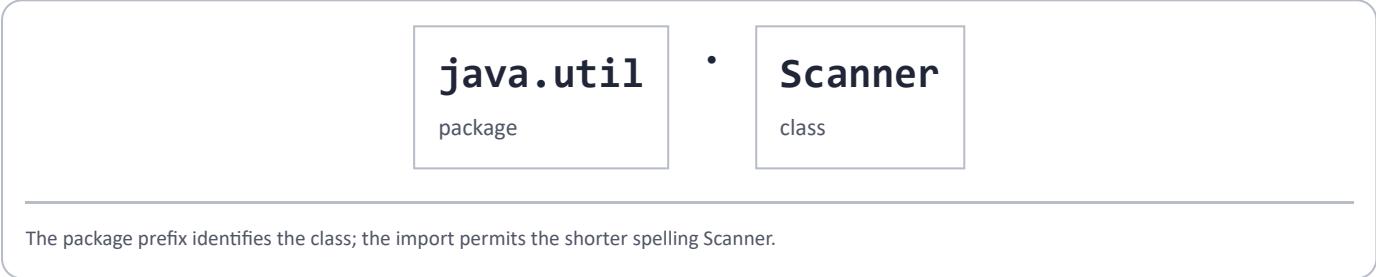
Package identity is below each root. src and classes are roots, not parts of the package name.

2. Definition and essential vocabulary

A Java package supplies a namespace for types. An import declaration lets a compilation unit refer to an accessible type using its simple name instead of its fully qualified name.



Term	Meaning in this chapter
Package	A namespace grouping related top-level types.
Import	A source-level name-shortening declaration.
Fully qualified name	Package plus type, such as <code>java.util.Scanner</code> .
Wildcard	Imports accessible types directly in that package; not its subpackages.
<code>java.lang</code>	Automatically imported; contains <code>String</code> , <code>System</code> and <code>Math</code> .



An import does not install a library, execute code or copy a class. A dependency still has to be available to compilation and runtime.

Package	Purpose	Examples
<code>java.lang</code>	Core language types, automatic import	<code>String</code> , <code>System</code> , <code>Math</code>
<code>java.util</code>	Utilities and collections	<code>Scanner</code> , <code>Arrays</code> , <code>ArrayList</code>
<code>java.io</code>	File and stream I/O	<code>File</code> , <code>BufferedReader</code> , <code>PrintWriter</code>

3. Visual workflow: declare, compile, run

Follow the same class through three representations

▼ Full trace table - select a row to jump to its scene

Step	Stage	Input	Output / resolution
1	Declare	<code>package pkg; / src/pkg/A.java</code>	<code>pkg.A</code>
2	Compile	<code>javac -d classes -sourcepath src ...</code>	<code>classes/pkg/*.class</code>

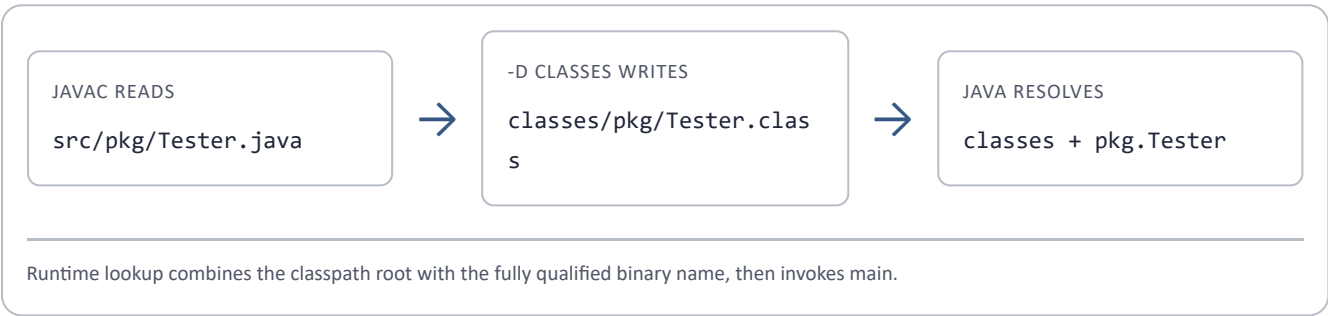
Step	Stage	Input	Output / resolution
3	Run	java -classpath classes pkg.Tester	classes/pkg/Tester.class → main

1. Declare identity



package pkg makes the public class A identifiable as pkg.A. The file belongs below source root src in pkg/A.java.

3. Resolve and run



java -classpath classes pkg.Tester locates classes/pkg/Tester.class and invokes its main method.

4. Rules before code

- Put a package declaration before import declarations and the class declaration.
- Match the package path below the source root.
- Match a public class name to its source filename, including case.
- Classes in the same package do not need imports for one another.
- Named packages cannot import a class from the unnamed default package.
- An import does not change an access modifier.
- Use a fully qualified class name when launching a packaged program.

Representation	Example	Root
Package	pkg	Not a filesystem root
Source	src/pkg/A.java	src
Compiled	classes/pkg/A.class	classes

Representation	Example	Root
Binary name	pkg.A	Root supplied separately by classpath

5. Fully commented Java and commands

JAVA

```
package app; // The package declaration precedes imports.

import java.util.Scanner; // Makes the simple name Scanner available.
import java.util.Arrays; // Imports one public type, not a whole library.

public class ImportDemo {
    public static void main(String[] args) {
        // System and String come from automatically imported java.lang.
        Scanner input = new Scanner(System.in);
        System.out.print("Enter one whole number: ");
        int value = input.nextInt(); // This example expects valid integer input.
        int[] values = {value, 2, 1};
        Arrays.sort(values); // Static utility method sorts in ascending order.
        System.out.println(Arrays.toString(values));
        // Equivalent spelling without a Scanner import:
        // java.util.Scanner input = new java.util.Scanner(System.in);
    }
}
```

Input 5 prints [1, 2, 5] after sorting. Save at src/app/ImportDemo.java.

COMMANDS

```
javac -d classes src/app/ImportDemo.java
java -classpath classes app.ImportDemo
```

Complete companion project for the package workflow

The following four source files make the earlier diagram and the later output/practice tables reproducible within this guide. Their package declarations and paths are explicit.

JAVA

```
package pkg; // Identity pkg.A; file src/pkg/A.java.

public class A { // Public class name must match A.java.
    // Other packages may call this public method on this public class.
    public static void greet() {
        System.out.println("Hello from pkg.A");
    }
}
```

JAVA

```
package pkg; // File src/pkg/B.java.

public class B {
    public static int twice(int x) { return 2 * x; }
    // No access modifier: only classes in package pkg may call secret.
    static String secret() { return "pkg only"; }
}
```

JAVA

```
package pkg; // File src/pkg/Tester.java.

public class Tester {
    // This exact main signature supplies the program entry point.
    public static void main(String[] args) {
        A.greet(); // Same package: no import required.
        System.out.println(B.twice(21)); // Prints 42.
        System.out.println(B.secret()); // Legal: Tester and B share pkg.
    }
}
```

JAVA

```
package app; // File src/app/Main.java.

import pkg.A; // Import a public class from another package.
import pkg.B;

public class Main {
    public static void main(String[] args) {
        A.greet();
        System.out.println(B.twice(5)); // Prints 10.
        // B.secret(); // Would fail: imports do not grant package-private access.
    }
}
```

COMMANDS

```
# Run from the project folder that contains src and classes.
javac -d classes -sourcepath src src/pkg/A.java src/pkg/B.java src/pkg/Tester.java src/app/Main.java

# Classpath points to the ROOT above pkg and app, not to classes/pkg.
java -classpath classes pkg.Tester
java -classpath classes app.Main
```

6. Visibility: imports and access are different

Member modifier	Same class	Same package	Subclass outside package	Unrelated outside class
public	Yes	Yes	Yes	Yes

Member modifier	Same class	Same package	Subclass outside package	Unrelated outside class
protected	Yes	Yes	Yes, with subclass access restrictions	No
none (package-private)	Yes	Yes	No	No
private	Yes	No	No	No

Protected access from a different package is through inheritance and is subject to receiver restrictions; it is not blanket permission to access any superclass object. A top-level class may be public or package-private, not protected/private.

ACCESS COMPARISON

```
import pkg.B;
// Legal in app.Main: B.twice(5)
// Illegal in app.Main: B.secret()
// Importing B does not alter secret() visibility.
```

7. Source-to-output trace table

Stage	Given	Resolution	Result
Declare	package pkg; / Tester.java	pkg.Tester	src/pkg/Tester.java
Compile	javac -d classes -sourcepath src ...	Write below output root	classes/pkg/Tester.class
Run	java -classpath classes pkg.Tester	Find class and invoke main	Hello from pkg.A / 42 / pkg only
Cross-package	java -classpath classes app.Main	Public APIs of pkg.A and pkg.B	Hello from pkg.A / 10

PKG.TESTER OUTPUT

```
Hello from pkg.A
42
pkg only
```

APP.MAIN OUTPUT

```
Hello from pkg.A
10
```

8. Five worked cases

1. Specific import

GIVEN

`import java.util.Scanner;`

→

RESOLUTION / RESULT

`Scanner in = new Scanner(System.in);`

The single imported type is available by its simple name.

Step	Evidence	Conclusion
1	<code>import java.util.Scanner;</code>	Identify package/type/root
2	<code>Scanner in = new Scanner(System.in);</code>	The single imported type is available by its simple name.

2. Fully qualified alternative

GIVEN

No Scanner import

→

RESOLUTION / RESULT

`java.util.Scanner in = new
java.util.Scanner(System.in);`

Same type; longer spelling resolves its identity directly.

Step	Evidence	Conclusion
1	No Scanner import	Identify package/type/root
2	<code>java.util.Scanner in = new java.util.Scanner(System.in);</code>	Same type; longer spelling resolves its identity directly.

3. Wildcard boundary

GIVEN

`import java.util.*;`

→

RESOLUTION / RESULT

Scanner works; a subpackage class still
needs its own import.

Wildcard imports do not recurse into subpackages.

Step	Evidence	Conclusion
1	<code>import java.util.*;</code>	Identify package/type/root
2	Scanner works; a subpackage class still needs its own import.	Wildcard imports do not recurse into subpackages.

4. Automatic names



Step	Evidence	Conclusion
1	No <code>import java.lang.String;</code>	Identify package/type/root
2	<code>String message = "Hello";</code>	java.lang types are automatically imported.

5. Ambiguous simple name



Step	Evidence	Conclusion
1	<code>import java.util.*; import java.sql.*;</code>	Identify package/type/root
2	<code>java.util.Date chosen = new java.util.Date();</code>	Both packages define Date; qualify the required type.

9. Build or explain it from scratch

- Identify the package and class you need.
- Decide specific import or fully qualified spelling.
- Keep package first, imports second, class third.

- Check that the imported type and used members are accessible.
- Compile a valid example; distinguish naming errors from visibility errors.

10. Properties and when to use packages

Packages organize names and collaboration; imports are compile-time name resolution conveniences. A wildcard is not a performance optimization. There is no useful sorting-style $O(n)$ claim to assign to an import declaration.

Property	Practical consequence
Namespace	Same simple type names may exist in different packages.
Encapsulation	Package-private members form an internal package API.
Maintainability	Related classes have predictable source locations.
Dependency lookup	Sourcepath/classpath roots must locate the required types.
Case sensitivity	pkg.A and pkg.a name different types.

11. Common errors and exact repairs

Message / symptom	Likely cause	Repair
cannot find symbol: Scanner	Simple name is not in scope.	Add import java.util.Scanner or use the fully qualified name.
reference to Date is ambiguous	Two wildcard-imported packages contain Date.	Use a specific import or fully qualified name.
package pkg does not exist	Compiler cannot locate sources/compiled dependencies.	Use -sourcepath src or -classpath classes as appropriate.
class A is public, should be declared in A.java	Public class and filename disagree.	Use A.java with public class A.
Could not find or load main class pkg.Tester	Wrong classpath root/name or missing compiled file.	Use classes as root and pkg.Tester without .class.
Main method not found in class pkg.A	That helper is not an entry-point class.	Run pkg.Tester or app.Main.
secret() is not public in B	Cross-package access to a package-private member.	Use a public API; an import cannot grant visibility.

12. Exam-style practice and answers

A. Place Helper in ub.cs.tools. Give declaration, source path, output path and binary name. [4]

package ub.cs.tools; / src/ub/cs/tools/Helper.java / classes/ub/cs/tools/Helper.class / ub.cs.tools.Helper. The src and classes roots are not package segments.

B. Explain import java.util.* and whether it includes subpackages. [3]

It allows accessible types directly in java.util to be named simply. It does not import types from subpackages, copy code or install dependencies.

C. Why can pkg.Tester call B.secret while app.Main cannot? [3]

secret has no access modifier, so it is package-private. Tester shares pkg. Main belongs to app; importing B does not grant member visibility.

D. Repair java -classpath classes/pkg Tester.class. [3]

Use java -classpath classes pkg.Tester. The classpath names the root above package folders; the launch argument is a dotted binary name without an extension.

E. Compile and run the four classes in this chapter. State expected output. [6]

COMMANDS

```
# Run from the project folder that contains src and classes.
javac -d classes -sourcepath src src/pkg/A.java src/pkg/B.java src/pkg/Tester.java src/app/Main.java

# Classpath points to the ROOT above pkg and app, not to classes/pkg.
java -classpath classes pkg.Tester
java -classpath classes app.Main
```

pkg.Tester prints Hello from pkg.A, 42, pkg only. app.Main prints Hello from pkg.A, 10.

F. Scanner is unresolved even though the JDK is installed. What are two fixes? [2]

Add import `java.util.Scanner`; or use `java.util.Scanner` at every relevant type/constructor use. Both name the same class.

13. Test-ready summary and mastery check

An import lets a source file use a simple type name; package declarations supply identity, and access modifiers supply visibility.

- I can separate package declaration, import and access modifier.
- I can translate dots to directories below a specified root.
- I can explain specific/wildcard/fully qualified alternatives.
- I can identify the main class and run it without an extension.
- I can diagnose naming, lookup and visibility errors separately.

CSI247 visual study guide · Built-in Packages and Imports · 2026-10-07.1

Structural reference: the author-supplied CSI247 guide, with topic-specific adaptation and repo-owned teaching content. All traces in this artifact are generated by deterministic simulations of its stated procedures. Counts distinguish key comparisons from loop tests, probes, swaps, shifts and array writes.